Name:Khushi Singh
Class:D20C Roll no.34

Experiment No. 3

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.
Guidelines

Theory:

1. Blockchain Overview

Blockchain is a distributed and decentralized ledger that stores information in a series of linked blocks.
Each block contains:

- Transaction data
- Timestamp
- Previous block's hash
- Its own unique hash (digital fingerprint)

Once data is recorded in a blockchain, it becomes immutable because altering one block would require recalculating all subsequent blocks.

2. Mining

Mining is the process of:

1. Collecting pending transactions into a block.
2. Performing a computational puzzle (Proof-of-Work) to find a valid hash.
3. Adding the new block to the blockchain.
   Broadcasting it to all connected peers.

Miners are rewarded with cryptocurrency for successfully mining a block.

3. Multi-Node Blockchain Network

In this lab, we simulate three independent blockchain nodes (5001, 5002, 5003).
Each node:

- Runs on a separate port.
- Maintains its own copy of the blockchain.
- Can connect with peers to share and validate blocks.

4. Consensus Mechanism

We use the Longest Chain Rule:

- If multiple versions of the chain exist, the longest valid chain is chosen.
- This ensures all nodes agree on a single transaction history.

5. Transactions & Mining Reward

Each transaction has:

- Sender
- Receiver
- Amount

When mining a block:

- Pending transactions are added to the block.
- A reward transaction is added automatically to pay the miner.

6. Chain Replacement

When /replace_chain is called:

1. Node requests chains from peers.
2. If it finds a longer and valid chain, it replaces its own.
3. This keeps the blockchain consistent across all nodes.

Tools & Libraries Used

- Python 3.x
- Flask – Web framework for API endpoints
  pip install Flask
- Requests – For HTTP communication between nodes
  pip install requests==2.18.4
- Postman – For testing API requests
- Python Standard Libraries:
  - datetime
  - jsonify
  - hashlib
  - uuid4
  - urlparse
  - request

Code:

```
# Module 2 - Create a Cryptocurrency
# To be installed:
# Flask==0.12.2: pip install
Flask==0.12.2
# Postman HTTP Client:
https://www.getpostman.com/
# requests==2.18.4: pip install
requests==2.18.4

# Importing the libraries
import datetime
import hashlib
import json
from flask import Flask, jsonify,
request
import requests
from uuid import uuid4
# Generate a unique id that is in
hex
from urllib.parse import urlparse
# To parse url of the nodes

# Part 1 - Building a Blockchain

class Blockchain:

    def __init__(self):
        self.chain = []
        self.transactions = []
# Adding transactions before they
are added to a block
        self.create_block(proof = 1,
previous_hash = '0')
        self.nodes = set()
# Set is used as there is no order
to be maintained as the nodes can be
from all around the globe

    def create_block(self, proof,
previous_hash):
        block = {'index':
len(self.chain) + 1,
                 'timestamp':
str(datetime.datetime.now()),
                 'proof': proof,
                 'previous_hash':
previous_hash,
                 'transactions':
self.transactions}
```

```
# Adding transactions to make the
blockchain a cryptocurrency
        self.transactions = []
# The list of transacction should
become empty after they are added to
a block
        self.chain.append(block)
        return block

    def get_previous_block(self):
        return self.chain[-1]

    def proof_of_work(self,
previous_proof):
        new_proof = 1
        check_proof = False
        while check_proof is False:
            hash_operation =
hashlib.sha256(str(new_proof**2 -
previous_proof**2).encode()).hexdige
st()
            if hash_operation[:4] ==
'0000':
                check_proof = True
            else:
                new_proof += 1
        return new_proof

    def hash(self, block):
        encoded_block =
json.dumps(block, sort_keys =
True).encode()
        return
hashlib.sha256(encoded_block).hexdig
est()

    def is_chain_valid(self, chain):
        previous_block = chain[0]
        block_index = 1
        while block_index <
len(chain):
            block =
chain[block_index]
            if
block['previous_hash'] !=
self.hash(previous_block):
                return False
            previous_proof =
previous_block['proof']
            proof = block['proof']
            hash_operation =
hashlib.sha256(str(proof**2 -
previous_proof**2).encode()).hexdige
st()
            if hash_operation[:4] !=
'0000':
                return False
            previous_block = block
            block_index += 1
        return True

    # This method will add the
trqnsaction to the list of
trnsactions
    def add_transaction(self,
sender, receiver, amount):

self.transactions.append({'sender':
sender,

'receiver': receiver,

'amount': amount})
        previous_block =
self.get_previous_block()
        return
previous_block['index'] + 1
# It will return the block index to
which the transaction should be
added

    # This function will add the
node containing an address to the
set of nodes created in init
function
    def add_node(self, address):
        parsed_url =
urlparse(address)
# urlparse will parse the url from
the address

self.nodes.add(parsed_url.netloc)
# Add is used and not append as it's
a set. Netloc will only return
'127.0.0.1:5000'

    # Consensus Protocol. This
function will replace all the
shorter chain with the longer chain
in all the nodes on the network
    def replace_chain(self):
        network = self.nodes
# network variable is the set of
nodes all around the globe
```

```
        longest_chain = None
# It will hold the longest chain
when we scan the network
        max_length = len(self.chain)
# This will hold the length of the
chain held by the node that runs
this function
        for node in network:
            response =
requests.get(f'http://{node}/get_cha
in')                 # Use get chain
method already created to  get the
length of the chain
            if response.status_code
== 200:
                length =
response.json()['length']
# Extract the length of the chain
from get_chain fiunction
                chain =
response.json()['chain']
                if length >
max_length and
self.is_chain_valid(chain):
# We check if the length is bigger
and if the chain is valid then
                    max_length =
length
# We update the max length
                    longest_chain =
chain
# We update the longest chain
        if longest_chain:
# If longest_chain is not none that
means it was replaced
            self.chain =
longest_chain
# Replace the chain of the current
node with the longest chain
            return True
        return False
# Return false if current chain is
the longest one

# Part 2 - Mining our Blockchain

# Creating a Web App
app = Flask(__name__)

# Creating an address for the node
on Port 5000. We will create some
other nodes as well on different
ports
node_address =
str(uuid4()).replace('-', '')
#

# Creating a Blockchain
blockchain = Blockchain()

# Mining a new block
@app.route('/mine_block', methods =
['GET'])
def mine_block():
    previous_block =
blockchain.get_previous_block()
    previous_proof =
previous_block['proof']
    proof =
blockchain.proof_of_work(previous_pr
oof)
    previous_hash =
blockchain.hash(previous_block)

blockchain.add_transaction(sender =
node_address, receiver = 'Richard',
amount = 1) # Hadcoins to mine the
block (A Reward). So the node gives
1 hadcoin to Abcde for mining the
block
    block =
blockchain.create_block(proof,
previous_hash)
    response = {'message':
'Congratulations, you just mined a
block!',
                'index':
block['index'],
                'timestamp':
block['timestamp'],
                'proof':
block['proof'],
                'previous_hash':
block['previous_hash'],
                'transactions':
block['transactions']}
    return jsonify(response), 200

# Getting the full Blockchain
@app.route('/get_chain', methods =
['GET'])
def get_chain():
```

```
    response = {'chain':
blockchain.chain,
                'length':
len(blockchain.chain)}
    return jsonify(response), 200

# Checking if the Blockchain is
valid
@app.route('/is_valid', methods =
['GET'])
def is_valid():
    is_valid =
blockchain.is_chain_valid(blockchain
.chain)
    if is_valid:
        response = {'message': 'All
good. The Blockchain is valid.'}
    else:
        response = {'message':
'Houston, we have a problem. The
Blockchain is not valid.'}
    return jsonify(response), 200

# Adding a new transaction to the
Blockchain
@app.route('/add_transaction',
methods = ['POST'])
# Post method as we have to pass
something to get something in return
def add_transaction():
    json = request.get_json()
# This will get the json file from
postman. In Postman we will create a
json file in which we will pass the
values for the keys in the json file
    transaction_keys = ['sender',
'receiver', 'amount']
    if not all(key in json for key
in transaction_keys):
# Checking if all keys are available
in json
        return 'Some elements of the
transaction are missing', 400
    index =
blockchain.add_transaction(json['sen
der'], json['receiver'],
json['amount'])
    response = {'message': f'This
transaction will be added to Block
{index}'}
    return jsonify(response), 201
# Code 201 for creation

# Part 3 - Decentralizing our
Blockchain

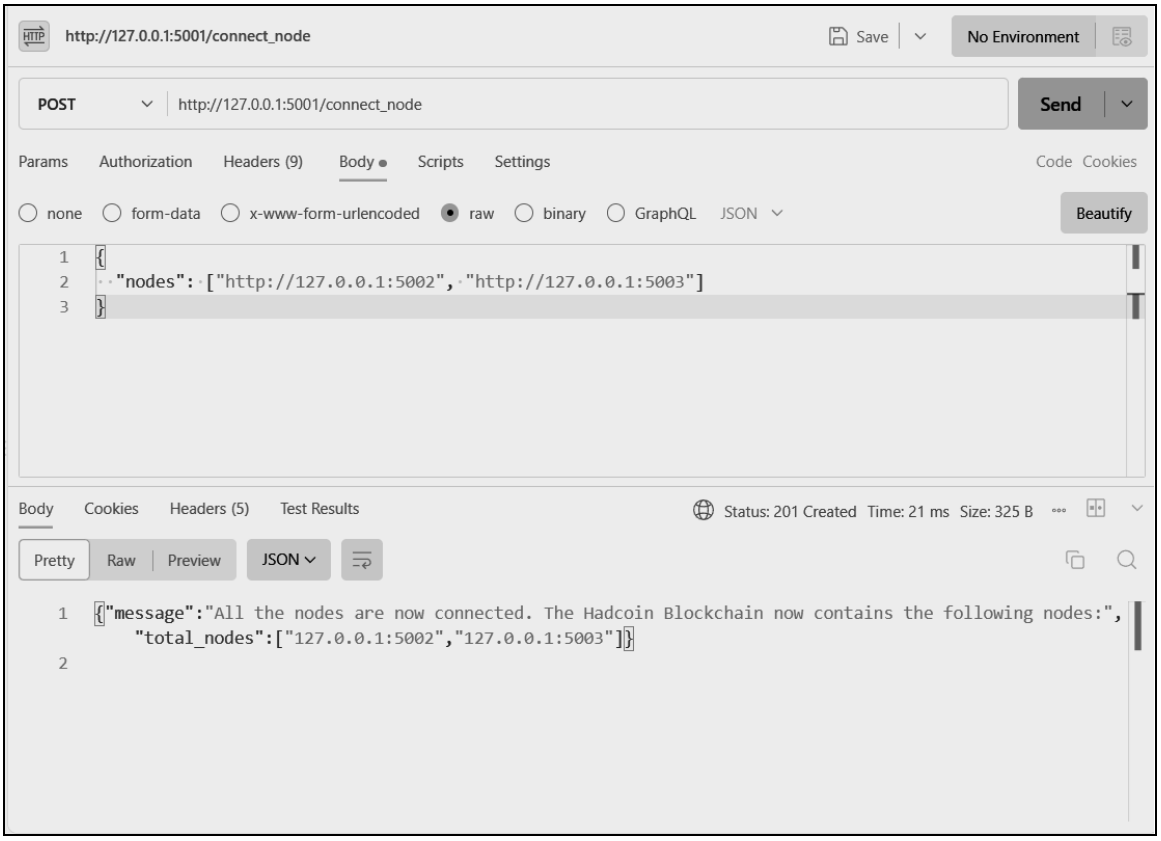
# Connecting new nodes
@app.route('/connect_node', methods
= ['POST'])
# POST request to register the new
nodes from the json file
def connect_node():
    json = request.get_json()
    nodes = json.get('nodes')
# Get the nodes from json file
    if nodes is None:
        return "No node", 400
    for node in nodes:
        blockchain.add_node(node)
    response = {'message': 'All the
nodes are now connected. The Hadcoin
Blockchain now contains the
following nodes:',
                'total_nodes':
list(blockchain.nodes)}
    return jsonify(response), 201

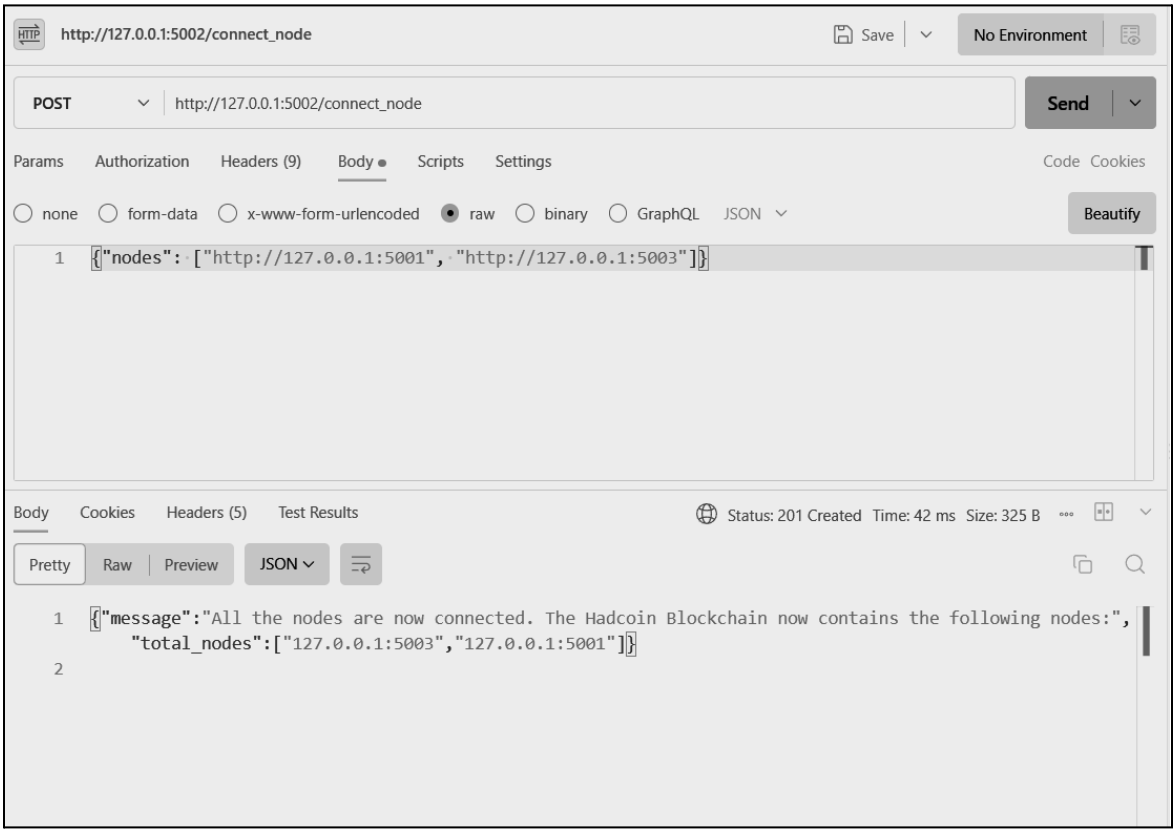
# Replacing the chain by the longest
chain if needed
@app.route('/replace_chain', methods
= ['GET'])
def replace_chain():
    is_chain_replaced =
blockchain.replace_chain()
    if is_chain_replaced:
        response = {'message': 'The
nodes had different chains so the
chain was replaced by the longest
one.',
                    'new_chain':
blockchain.chain}
    else:
        response = {'message': 'All
good. The chain is the largest
one.',
                    'actual_chain':
blockchain.chain}
    return jsonify(response), 200

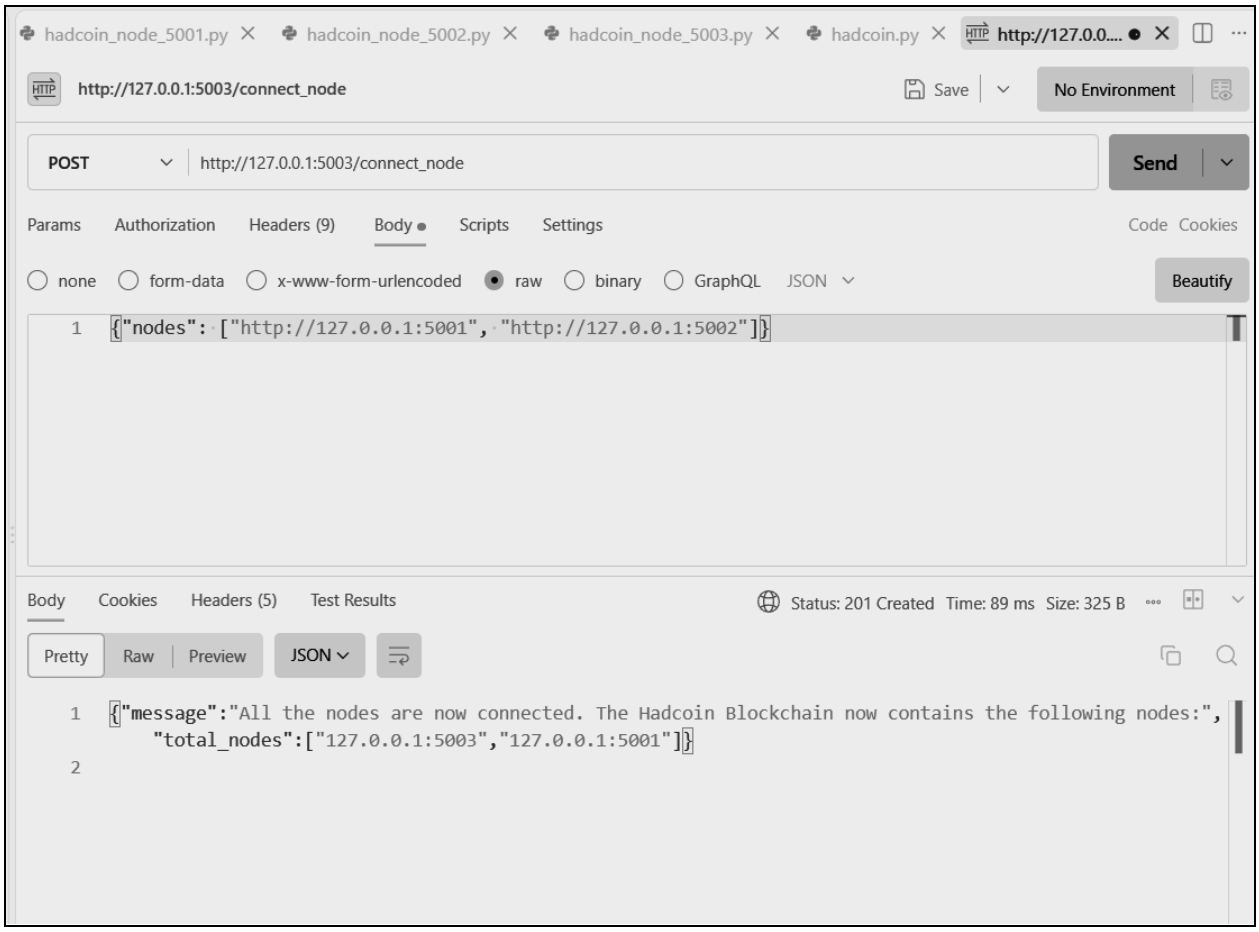
# Running the app
app.run(host = '0.0.0.0', port =
5001)
```

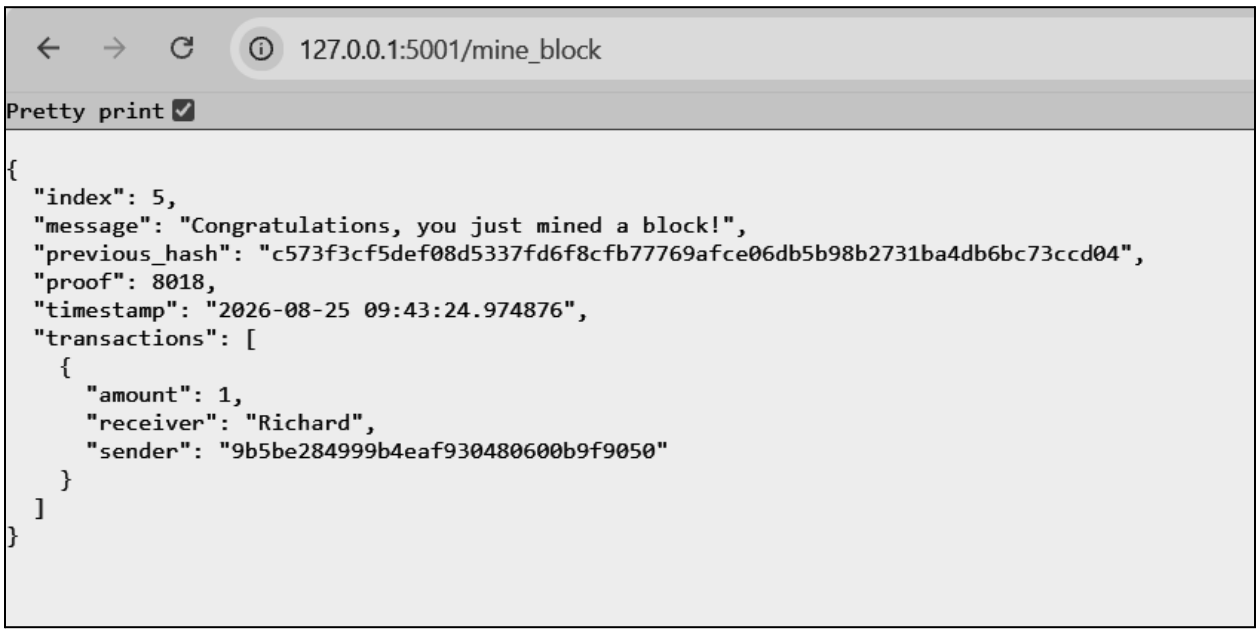
Procedure and Output:

1. Download hadcoin_node_5001.py, hadcoin_node_5002.py, hadcoin_node_5003.py.
2. Install required packages.
3. Run each node in separate terminals.


http://127.0.0.1:5001/connect_node
Save
No Environment
POST
http://127.0.0.1:5001/connect_node
Send
Params
Authorization
Headers (9)
Body
Scripts
Settings
Code
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
{
"nodes": ["http://127.0.0.1:5002", "http://127.0.0.1:5003"]
}
Body
Cookies
Headers (5)
Test Results
Status: 201 Created
Time: 21 ms
Size: 325 B
Pretty
Raw
Preview
JSON
{"message":"All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
"total_nodes":["127.0.0.1:5002","127.0.0.1:5003"]}


http://127.0.0.1:5002/connect_node
Save
No Environment
POST
http://127.0.0.1:5002/connect_node
Send
Params
Authorization
Headers (9)
Body
Scripts
Settings
Code
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
{"nodes": ["http://127.0.0.1:5001", "http://127.0.0.1:5003"]}
Body
Cookies
Headers (5)
Test Results
Status: 201 Created
Time: 42 ms
Size: 325 B
Pretty
Raw
Preview
JSON
{"message":"All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
"total_nodes":["127.0.0.1:5003","127.0.0.1:5001"]}


hadcoin_node_5001.py
hadcoin_node_5002.py
hadcoin_node_5003.py
hadcoin.py
http://127.0.0....
http://127.0.0.1:5003/connect_node
Save
No Environment
POST
http://127.0.0.1:5003/connect_node
Send
Params
Authorization
Headers (9)
Body
Scripts
Settings
Code
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
1 {"nodes": ["http://127.0.0.1:5001", "http://127.0.0.1:5002"]}
Body
Cookies
Headers (5)
Test Results
Status: 201 Created
Time: 89 ms
Size: 325 B
Pretty
Raw
Preview
JSON
1 {"message":"All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
"total_nodes":["127.0.0.1:5003","127.0.0.1:5001"]}
2


127.0.0.1:5001/mine_block
Pretty print
{
"index": 5,
"message": "Congratulations, you just mined a block!",
"previous_hash": "c573f3cf5def08d5337fd6f8cfb77769afce06db5b98b2731ba4db6bc73ccd04",
"proof": 8018,
"timestamp": "2026-08-25 09:43:24.974876",
"transactions": [
{
"amount": 1,
"receiver": "Richard",
"sender": "9b5be284999b4eaf930480600b9f9050"
}
]
}

Pretty print ☑


```
{
  "actual_chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-08-25 09:14:32.926683",
      "transactions": []
    },
    {
      "index": 2,
      "previous_hash": "098e018a30edf3875a3d674b9cc44c3241f73e9851abb9d35ccc2f9ff10a1215",
      "proof": 533,
      "timestamp": "2026-08-25 09:28:50.753890",
      "transactions": [
        {
          "amount": 1,
          "receiver": "Richard",
          "sender": "9b5be284999b4eaf930480600b9f9050"
        }
      ]
    }
  ],
  "message": "All good. The chain is the largest one."
}
```

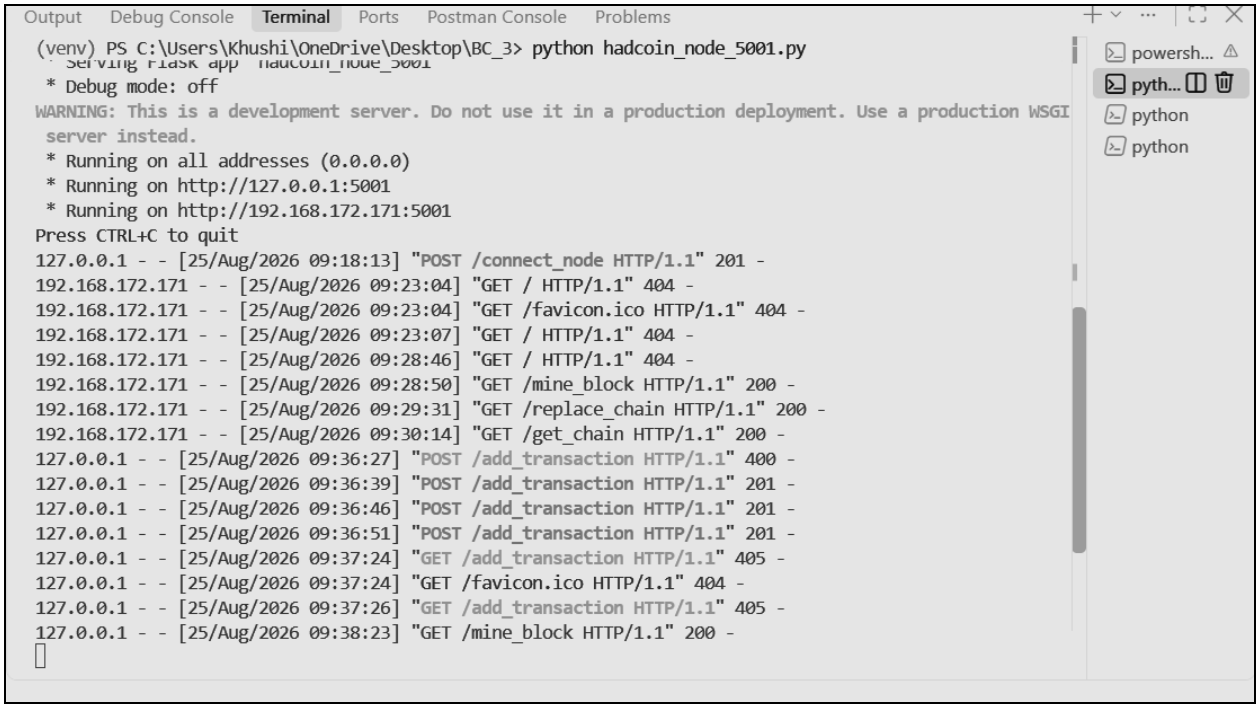

Pretty print ☑

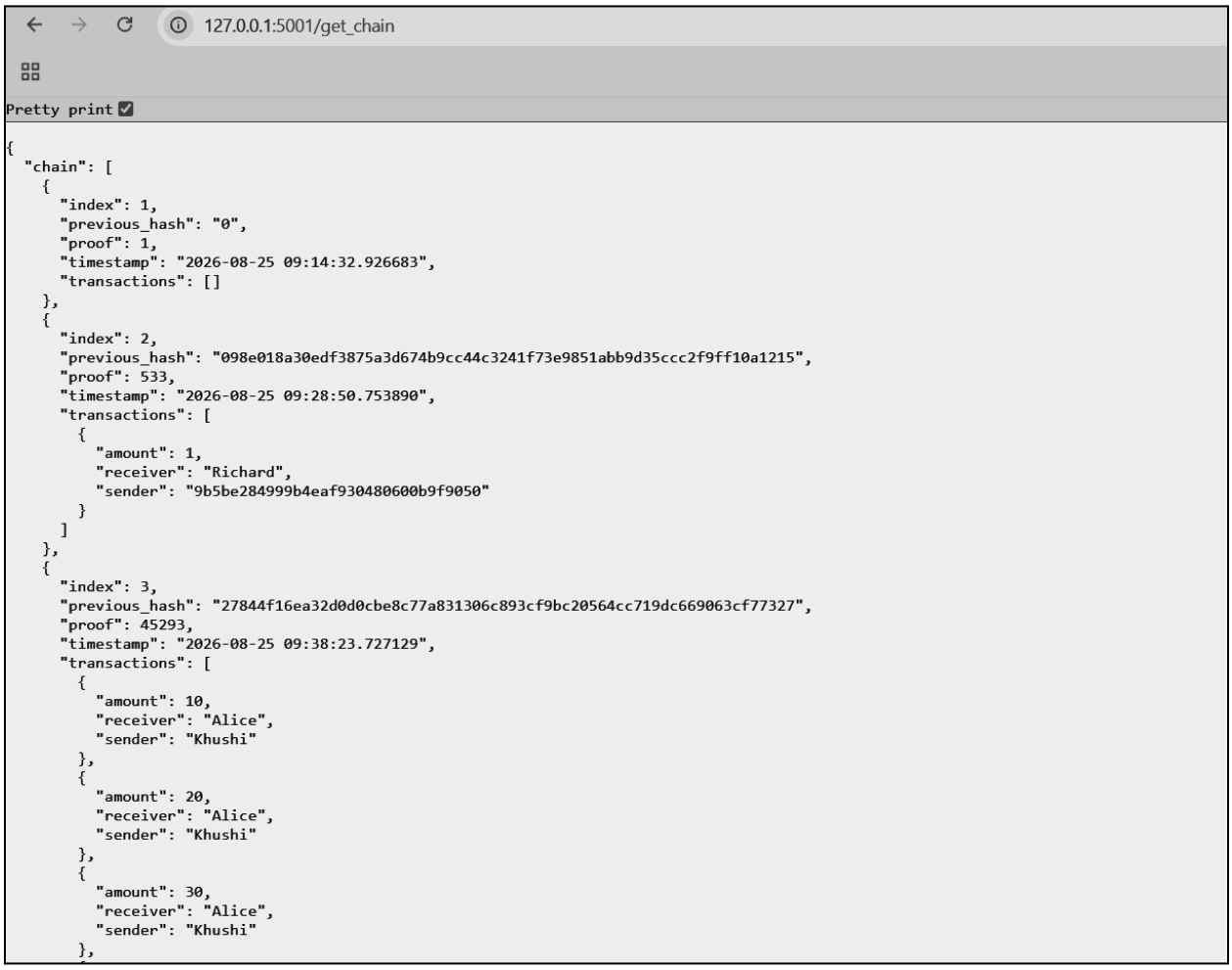

```
{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-08-25 09:14:32.926683",
      "transactions": []
    },
    {
      "index": 2,
      "previous_hash": "098e018a30edf3875a3d674b9cc44c3241f73e9851abb9d35ccc2f9ff10a1215",
      "proof": 533,
      "timestamp": "2026-08-25 09:28:50.753890",
      "transactions": [
        {
          "amount": 1,
          "receiver": "Richard",
          "sender": "9b5be284999b4eaf930480600b9f9050"
        }
      ]
    }
  ],
  "length": 2
}
```

```
(venv) PS C:\Users\Khushi\OneDrive\Desktop\BC_3> python hadcoin_node_5001.py
    )
ModuleNotFoundError: No module named 'urllib3.packages.six.moves'
(venv) PS C:\Users\Khushi\OneDrive\Desktop\BC_3> python hadcoin_node_5001.py
 * Serving Flask app 'hadcoin_node_5001'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI
 server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5001
 * Running on http://192.168.172.171:5001
Press CTRL+C to quit
127.0.0.1 - - [25/Aug/2026 09:18:13] "POST /connect_node HTTP/1.1" 201 -
192.168.172.171 - - [25/Aug/2026 09:23:04] "GET / HTTP/1.1" 404 -
192.168.172.171 - - [25/Aug/2026 09:23:04] "GET /favicon.ico HTTP/1.1" 404 -
192.168.172.171 - - [25/Aug/2026 09:23:07] "GET / HTTP/1.1" 404 -
192.168.172.171 - - [25/Aug/2026 09:28:46] "GET / HTTP/1.1" 404 -
192.168.172.171 - - [25/Aug/2026 09:28:50] "GET /mine_block HTTP/1.1" 200 -
192.168.172.171 - - [25/Aug/2026 09:29:31] "GET /replace_chain HTTP/1.1" 200 -
192.168.172.171 - - [25/Aug/2026 09:30:14] "GET /get_chain HTTP/1.1" 200 -
```

Pretty print ☑

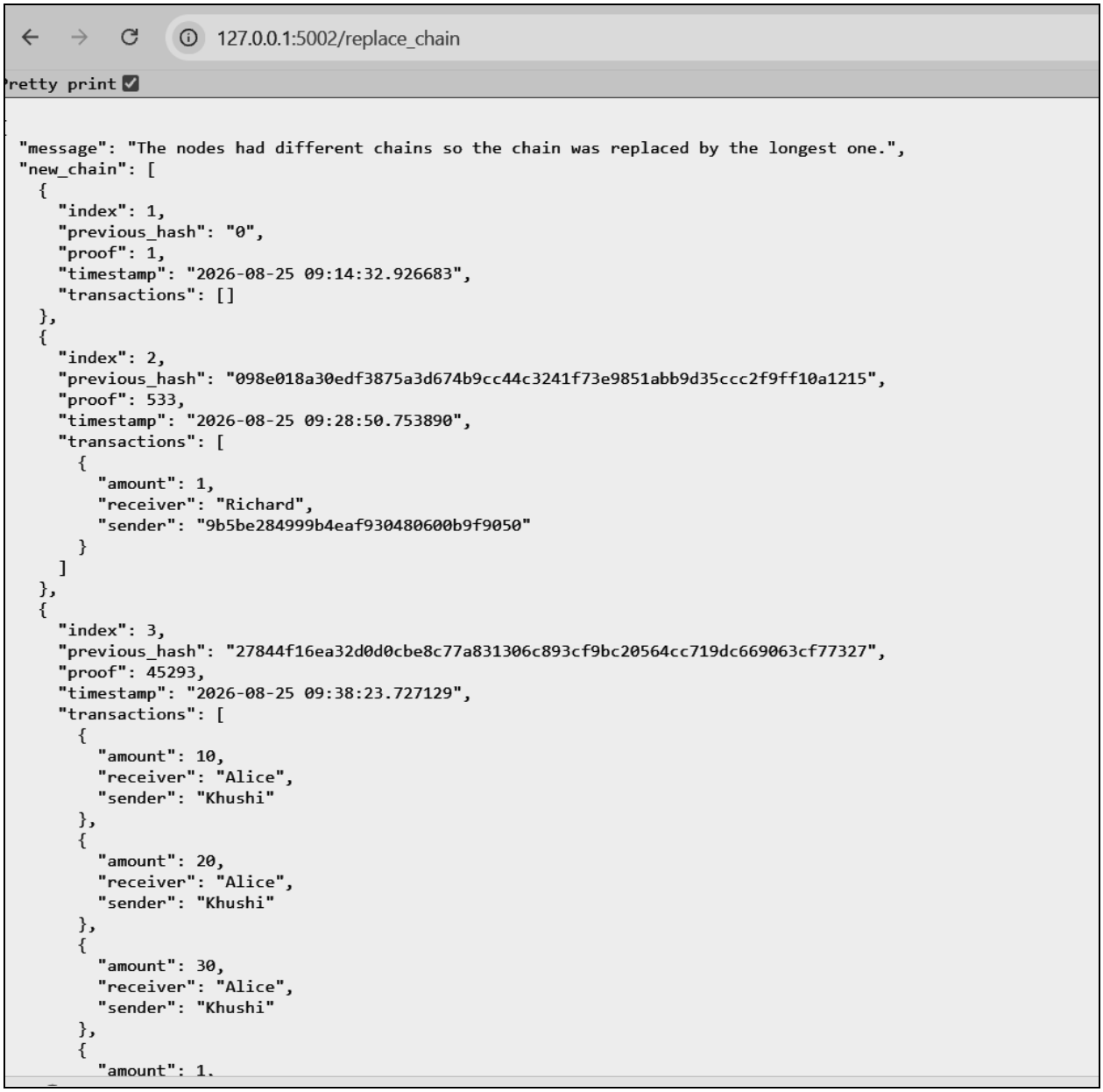

```
{
  "index": 3,
  "message": "Congratulations, you just mined a block!",
  "previous_hash": "27844f16ea32d0d0cbe8c77a831306c893cf9bc20564cc719dc669063cf77327",
  "proof": 45293,
  "timestamp": "2026-08-25 09:38:23.727129",
  "transactions": [
    {
      "amount": 10,
      "receiver": "Alice",
      "sender": "Khushi"
    },
    {
      "amount": 20,
      "receiver": "Alice",
      "sender": "Khushi"
    },
    {
      "amount": 30,
      "receiver": "Alice",
      "sender": "Khushi"
    },
    {
      "amount": 1,
      "receiver": "Richard",
      "sender": "9b5be284999b4eaf930480600b9f9050"
    }
  ]
}
```


Output
Debug Console
Terminal
Ports
Postman Console
Problems
(venv) PS C:\Users\Khushi\OneDrive\Desktop\BC_3> python hadcoin_node_5001.py
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5001
* Running on http://192.168.172.171:5001
Press CTRL+C to quit
127.0.0.1 - - [25/Aug/2026 09:18:13] "POST /connect_node HTTP/1.1" 201 -
192.168.172.171 - - [25/Aug/2026 09:23:04] "GET / HTTP/1.1" 404 -
192.168.172.171 - - [25/Aug/2026 09:23:04] "GET /favicon.ico HTTP/1.1" 404 -
192.168.172.171 - - [25/Aug/2026 09:23:07] "GET / HTTP/1.1" 404 -
192.168.172.171 - - [25/Aug/2026 09:28:46] "GET / HTTP/1.1" 404 -
192.168.172.171 - - [25/Aug/2026 09:28:50] "GET /mine_block HTTP/1.1" 200 -
192.168.172.171 - - [25/Aug/2026 09:29:31] "GET /replace_chain HTTP/1.1" 200 -
192.168.172.171 - - [25/Aug/2026 09:30:14] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [25/Aug/2026 09:36:27] "POST /add_transaction HTTP/1.1" 400 -
127.0.0.1 - - [25/Aug/2026 09:36:39] "POST /add_transaction HTTP/1.1" 201 -
127.0.0.1 - - [25/Aug/2026 09:36:46] "POST /add_transaction HTTP/1.1" 201 -
127.0.0.1 - - [25/Aug/2026 09:36:51] "POST /add_transaction HTTP/1.1" 201 -
127.0.0.1 - - [25/Aug/2026 09:37:24] "GET /add_transaction HTTP/1.1" 405 -
127.0.0.1 - - [25/Aug/2026 09:37:24] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [25/Aug/2026 09:37:26] "GET /add_transaction HTTP/1.1" 405 -
127.0.0.1 - - [25/Aug/2026 09:38:23] "GET /mine_block HTTP/1.1" 200 -
powersh...
pyth...
python
python


127.0.0.1:5001/get_chain
Pretty print
{
"chain": [
{
"index": 1,
"previous_hash": "0",
"proof": 1,
"timestamp": "2026-08-25 09:14:32.926683",
"transactions": []
},
{
"index": 2,
"previous_hash": "098e018a30edf3875a3d674b9cc44c3241f73e9851abb9d35ccc2f9ff10a1215",
"proof": 533,
"timestamp": "2026-08-25 09:28:50.753890",
"transactions": [
{
"amount": 1,
"receiver": "Richard",
"sender": "9b5be284999b4eaf930480600b9f9050"
}
]
},
{
"index": 3,
"previous_hash": "27844f16ea32d0d0cbe8c77a831306c893cf9bc20564cc719dc669063cf77327",
"proof": 45293,
"timestamp": "2026-08-25 09:38:23.727129",
"transactions": [
{
"amount": 10,
"receiver": "Alice",
"sender": "Khushi"
},
{
"amount": 20,
"receiver": "Alice",
"sender": "Khushi"
},
{
"amount": 30,
"receiver": "Alice",
"sender": "Khushi"
},


127.0.0.1:5002/get_chain
Pretty print
{
"chain": [
{
"index": 1,
"previous_hash": "0",
"proof": 1,
"timestamp": "2026-08-25 09:14:57.526185",
"transactions": []
}
],
"length": 1
}


127.0.0.1:5003/get_chain
Pretty print
{
"chain": [
{
"index": 1,
"previous_hash": "0",
"proof": 1,
"timestamp": "2026-08-25 09:15:08.112906",
"transactions": []
}
],
"length": 1
}


127.0.0.1:5002/replace_chain
Pretty print
{
"message": "The nodes had different chains so the chain was replaced by the longest one.",
"new_chain": [
{
"index": 1,
"previous_hash": "0",
"proof": 1,
"timestamp": "2026-08-25 09:14:32.926683",
"transactions": []
},
{
"index": 2,
"previous_hash": "098e018a30edf3875a3d674b9cc44c3241f73e9851abb9d35ccc2f9ff10a1215",
"proof": 533,
"timestamp": "2026-08-25 09:28:50.753890",
"transactions": [
{
"amount": 1,
"receiver": "Richard",
"sender": "9b5be284999b4eaf930480600b9f9050"
}
]
},
{
"index": 3,
"previous_hash": "27844f16ea32d0d0cbe8c77a831306c893cf9bc20564cc719dc669063cf77327",
"proof": 45293,
"timestamp": "2026-08-25 09:38:23.727129",
"transactions": [
{
"amount": 10,
"receiver": "Alice",
"sender": "Khushi"
},
{
"amount": 20,
"receiver": "Alice",
"sender": "Khushi"
},
{
"amount": 30,
"receiver": "Alice",
"sender": "Khushi"
},
{
"amount": 1,

127.0.0.1:5003/replace_chain

Pretty print


```
{
  "message": "The nodes had different chains so the chain was replaced by the longest one.",
  "new_chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-08-25 09:14:32.926683",
      "transactions": []
    },
    {
      "index": 2,
      "previous_hash": "098e018a30edf3875a3d674b9cc44c3241f73e9851abb9d35ccc2f9ff10a1215",
      "proof": 533,
      "timestamp": "2026-08-25 09:28:50.753890",
      "transactions": [
        {
          "amount": 1,
          "receiver": "Richard",
          "sender": "9b5be284999b4eaf930480600b9f9050"
        }
      ]
    },
    {
      "index": 3,
      "previous_hash": "27844f16ea32d0d0cbe8c77a831306c893cf9bc20564cc719dc669063cf77327",
      "proof": 45293,
      "timestamp": "2026-08-25 09:38:23.727129",
      "transactions": [
        {
          "amount": 10,
          "receiver": "Alice",
          "sender": "Khushi"
        },
        {
          "amount": 20,
          "receiver": "Alice",
          "sender": "Khushi"
        },
        {
          "amount": 30,
          "receiver": "Alice",
          "sender": "Khushi"
        },
        {
          "amount": 1,
```








Conclusion:

We developed a cryptocurrency using Python and implemented mining in a simulated three-node blockchain network. Each node maintained its own ledger and synchronized with peers using the Longest Chain Rule to ensure consistency. Mining was performed through Proof-of-Work, securely adding transactions and rewarding miners. This demonstrated key blockchain concepts, including decentralized consensus, transaction validation, and network synchronization.